



PURBANCHAL UNIVERSITY

KULESHWOR AWAS CAMPUS

SECOND SEMESTER PROJECT

ON

“SMART FARE SIMULATION”

In the partial fulfilments for the requirement of the first Semester Project-II (Subject code-BIT156CO) in the completion of Bachelor of Information Technology (BIT) degree at Kuleshwor Awas Campus, under Purbanchal University.

Submitted By:

Name: Pasang Dorje Tamang

S.N:

Name: Prasanna Katuwal

S.N:

Submitted To:

Purbanchal University

2025-6-9

ACKNOWLEDGEMENT

We take this opportunity to express our profound appreciation and unfathomable regards to the Information Technology (IT) department for this commendable guidance, monitoring and constant encouragement throughout the course of this project. The help and guidance given by shall carry us a long way, in the journey on which we are about to embark.

We also take this opportunity to express a deep sense of gratefulness to Mr Narayan GC for his amiable support, valuable information and guidance, which helped us in completing this task throughout its various stages. We also want to thank our program director Mr Narayan G.C. who helped us a lot during the complete project by giving us his precious time. We are also indebted to all members of Kuleshwor Awas Campus, for the help provided by them in their in their respective fields. We are grateful for their cooperation during the period of our project.

Finally we would also like to express lots of thanks to PURBANCHAL UNIVERSITY for designing such a wonderful course structure. It will help us to get more knowledge in the field of Information Technology & help us to have a bright future in the field of technology.

STUDENT'S DECLARATION

We following students, hereby declare that the Project Report is titled

“Smart Fare Simulation” is a result of our own work and our indebtedness to other work publications, references, if any, have been dully acknowledged. If we are found guilty of copying any other report or published information and showing as our original work, we understand that we shall be liable and punishable by Purbanchal

University, which may include fail in examination, ‘Repeat study and re-submission of the report’ or any other punishment the Purbanchal University may decide.

We further certify that this Project submitted in partial fulfilment of the requirement for the award of Bachelor in Information Technology (BIT) of the Purbanchal University is our original work and has not been submitted for award of any other degree or other similar title or prizes.

S.N	Name	Redg	Roll no.	Signature
1.	Pasang Dorje Tamang	177-3-2-01980-2024		
2.	Prasanna Katuwal			



Purbanchal University

Office of the Examination Management, Biratnagar

EXAMINER'S CERTIFICATION

This is to certify that the project entitled “Smart Fare Simulation” has been successfully completed by Mr Pasang Dorje Tamang (S/N:.....) & Mr Prasanna Katuwal (S/N:.....) in partial fulfilment of Degree of Bachelor of Information Technology of Purbanchal University during the academic year 2016 under the guidance of Mr Narayan G.C.

.....
Department of Science and Technology
Kuleshwor Awas Campus
PURBANCHAL UNIVERSITY

To Whom It May Concern

This is to clarify that Mr Pasang Dorje Tamang and Mr Prasanna Katuwal, students of Bachelor in information technology (BIT) program, have successfully completed their second-semester project named “Smart Fare Simulation”.

This project is the original work of Mr Pasang Dorje Tamang and Mr Prasanna Katuwal, carried out under the supervision of Mr Narayan G.C. as per guideline provided by Purbanchal University, and is certified as per the student’s declaration that the project “Smart Fare Simulation” has not been submitted or presented elsewhere as part of any academic work.

The details of the student are as follows:-

Name of student: -Pasang Dorje Tamang, Prasanna Katuwal

Course Semester: - Second Semester

Subject: - Project-II

Subject Code: - BIT156CO

.....
Narayan G.C.
Program Director, BIT
KULESHWOR AWAS CAMPUS

ABSTRACT

The Smart Fare Simulation is a console-based application developed in C++ that allows users to manage public transportation cards efficiently. Users can register new cards, choose card types such as normal, student, or elder citizen, and receive automatic fare discounts based on card type.

The system enables users to securely top up their card balances, start rides by selecting stops, and automatically calculates and deducts fares. Ride histories are stored and can be reviewed at any time, providing transparency and convenience for regular commuters.

An admin panel allows for card and ride management, including viewing all cards, updating balances, and blocking or unblocking cards. All data is stored in CSV files, ensuring persistence across sessions. The project demonstrates fundamental programming concepts such as file handling, modular design, input validation, and basic data processing.

Table of Contents

ACKNOWLEDGEMENT	II
STUDENT'S DECLARATION	III

EXAMINER’S CERTIFICATION	IV
ABSTRACT	V
List of Figures.....	VII
LIST OF ABBREVIATIONS.....	VIII
Chapter 1: Introduction.....	1
1.1 INTRODUCTION	1
1.2 OBJECTIVE OF THIS PROJECT	1
1.3 METHODOLOGY	2
1.4 SOFTWARE / LANGUAGE USED	3
1.5 TIMELINE.....	3
Chapter 2: SYSTEM REQUIREMENT	4
Chapter 3: SYSTEM DESIGN	4
3.1 INTRODUCTION TO THE PROJECT	5
3.2 ALGORITHM.....	5
3.3 FLOWCHART	7
3.4 SYSTEM REQUIREMENT	8
3.4.1 FUNCTIONAL REQUIREMENT	8
3.4.2 NON-FUNCTIONAL REQUIREMENT	12
CHAPTER 4: CONCLUSION	13
4.1 PROBLEM FACED.....	13
4.2 LESSON LEARNED	13
4.3 CONCLUSION.....	13
4.4 RECOMMENDATION	14
4.5 FUTURE ENHANCEMENT.....	14
BIBLIOGRAPHY.....	

List of Figures

Fig.	Name	Page no.
1.3	Incremental Approach	2
1.5	Gantt chart	3

3.3	Flow chart	7
3.4.1.1	Main menu	8
3.4.1.3	Calendar view	8
3.4.1.3	Date validation	9
3.4.1.4	To-Do list menu	9
3.4.1.5	View To-Do list	9
3.4.1.6	Mart To-Do as complete	10

LIST OF ABBREVIATIONS

BIT	Bachelor of Information Technology
-----	------------------------------------

C	(Programming Language) C
GCC	GNU Compiler Collection
IDE	Integrated Development Environment
PC	Personal Computer
S/N	Serial Number
UI	User Interface

Chapter 1: Introduction

1.1 INTRODUCTION

Efficient management of public transportation is essential in modern urban life, where commuters often rely on multiple rides daily. Tracking fares, card balances, and ride history manually can be confusing and time-consuming. The Smart Fare Simulation provides a digital solution to streamline this process, making commuting more organized and convenient.

This system allows users to register transport cards, select card types such as normal, student, or elder citizen, and automatically apply discounts while calculating fares. Users can securely top up balances, start rides by selecting stops, and keep track of ride histories, ensuring transparency and ease of use.

Additionally, the system includes an admin panel for overseeing all cards and rides, updating balances, and managing blocked or inactive cards. By digitizing fare management and record-keeping, this project helps commuters save time, avoid errors, and manage their travel efficiently, while demonstrating fundamental programming concepts such as file handling, modular design, and data processing

1.2 OBJECTIVE OF THIS PROJECT

- To provide easy way to manage transport cards, fares, and rides
- To allow card registration, top-up balance, and automatic fare calculations.
- To track ride history and apply discounts for students and elder citizens.
- To enable admin management of cards, balance and blocked users.
- To store data efficiently in CSV files for persistence and easy access.

1.3 METHODOLOGY

The development of the Smart fare Simulation followed a modular and structured approach aimed at creating a lightweight, user-friendly, and efficient scheduling tool using the C++ programming language.

The first phase involved requirement analysis and planning, where essential features like Card Registration, balance top-up, admin panel and fare calculation were identified. Decisions regarding file-based storage for persistence and a console-based user interface were finalized to ensure simplicity and portability.

In the second phase, design and implementation, the application was broken down into modular C++ functions for each core feature. Special focus was given to accurate fare management for corresponding card type, file handling for storing rides history and cards, and validating user inputs for correctness and reliability.

The final phase was testing and refinement, where individual modules were tested for edge cases such as invalid amount, duplicate entries, and file access errors. Based on test results and user experience, improvements like clear prompts, input confirmations, and structured menus were added to enhance usability and system stability.

This step-by-step methodology ensured the system was robust, practical, and met the intended goals for time and task management.

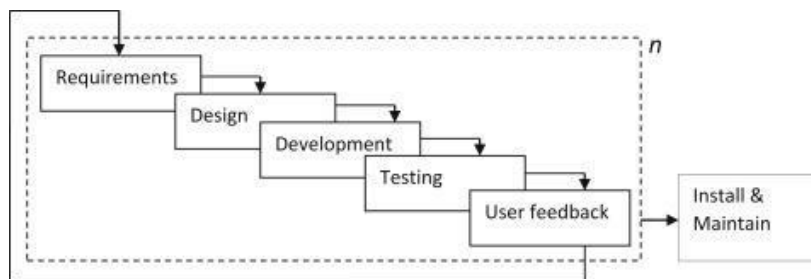


Figure 1.3 Incremental Approach

1.4 SOFTWARE / LANGUAGE USED

C++ is a powerful general-purpose programming language developed as an extension of C, adding object-oriented features and other modern programming capabilities. It is widely used for system programming, application development, and projects that require efficient data handling and modular design. C++ combines low-level memory manipulation with high-level abstractions, making it ideal for developing software like the Smart Fare Simulation, which involves managing cards, fares, rides, and user data efficiently.

1.5 TIMELINE

The project was developed over a period of 2 to 3 months and was divided into clear, structured phases to ensure smooth progress. The timeline outlines each stage of development, starting from idea selection and requirement analysis, moving through coding and implementation of features like the calendar, event manager, and to-do list, and ending with testing, documentation, and final submission. This organized progression helped maintain focus, track milestones, and ensure that each module was completed effectively before moving on to the next.

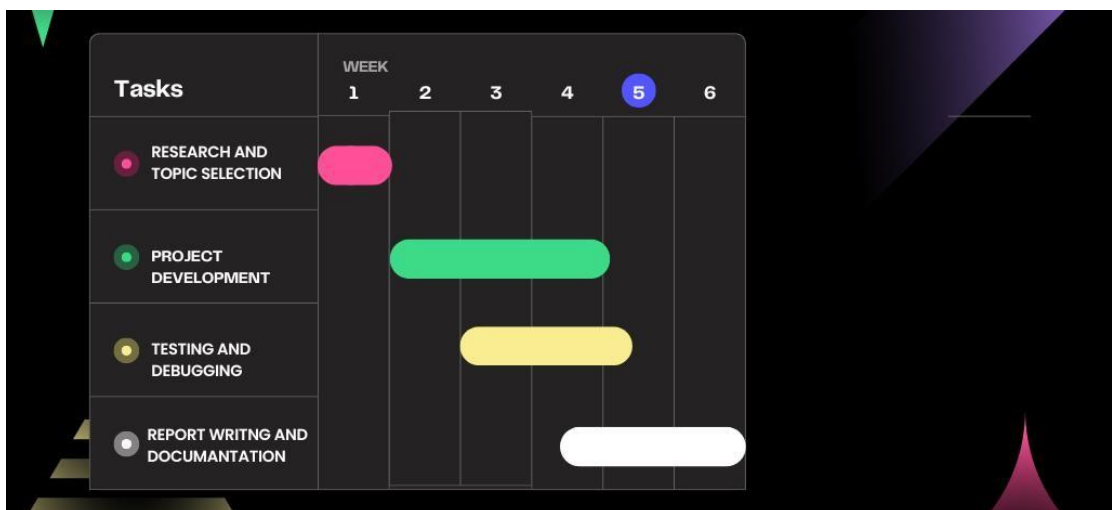


Figure 1.5 Gantt chart

Chapter 2: SYSTEM REQUIREMENT

2.1 HARDWARE

To ensure smooth development and execution of the project, the following hardware components were required and used during the implementation phase.

- PC with Pentium II Processor (260 MHz) or higher
- 32 MB RAM or more
- Color Monitor (preferred)
- Hard disk with at least 50 MB of free space

2.2 SOFTWARE

The successful development and execution of the project relied on specific software tools and environments that supported coding, compilation, and testing of the system.

- The project requires a C compiler such as GCC or Turbo C to compile and execute the source code.
- An Integrated Development Environment (IDE) like Code::Blocks, Dev-C++, or Visual Studio Code can be used to write and manage the code more efficiently.
- The system is compatible with Windows, Linux, or macOS operating systems.
- A text editor is needed to view and modify output files such as event.txt and todo.txt.

Chapter 3: SYSTEM DESIGN

3.1 INTRODUCTION TO THE PROJECT

This project is a Smart Fare Simulation developed in the C plus plus programming language. It provides users with a simple and efficient way to manage fare using cards, manage their ride history, and commute daily without any stress of having change money. The system ensures data persistence using file handling and offers a user-friendly menu driven interface for easy navigation.

3.2 ALGORITHM

The algorithm for the Smart fare simulation is designed to handle fare calculations, trip management, and ride tracking efficiently. It starts by allowing the user to register new cards. For ride management, the system allows users to register card, top-up cards and track rides while ensuring all validations. The admin panel supports all tasks that user can do from admin side allowing more like blocking and unblocking card. File handling is used throughout to store and retrieve data, ensuring persistence across sessions. The algorithm is implemented through modular functions to maintain clarity and ease of debugging.

The system consist of different sets of algorithms:

Step 1: Display main menu with options:

- Register new card
- Top-up card balance
- Start a ride
- View Card Details
- View Ride history
- Admin Panel (only admin can access)
- Exit Program

Step 2: Wait for user input (1–7) and process based on selected option.

Step 3: If Option 1 (Register new Card):

- Ask for Full name and Card type (student, elder citizen, normal).
- After validating Card register successfully and saved to card.csv.

Step 4: If Option 2 (Top-up card balance):

- Ask for user's card id.
- Ask amount and to enter password
- Balance update successfully.

Step 5: If Option 3 (Start a Ride):

- Ask to enter card ID.
- After validating card ID asks starting location and ending location
- Fare is deducted and balance update incard.csv.

Step 6: If Option 4 (View Card Details):

- Ask for specific card ID.
- Find the card detail from card.csv and display details.

Step 7: If Option 5 (View Ride History):

- Ask for specific card ID.
- Find the ride history from ride_history.csv and display ride history.

Step 8: If Option 6 (Admin Panel):

Display submenu:

1. View all cards
2. Search card by ID
3. Update balance
4. Block Card
5. Unblock Card
6. Exit Admin Panel

Each sub-option performs corresponding operations.

Step 8: If Option 7 (Exit):

- Terminate program using.

Step 9: Stop

3.3 FLOWCHART

The flowchart represents the logical structure and sequence of operations in the Smart fare simulations. It begins with the main menu, allowing users to choose between register card, top-up balance and start and save ride. Each option leads to a specific set of operations such as adding, viewing, or deleting data. Decision-making blocks guide the program flow based on user input, while arrows indicate the transition between different processes. The flowchart helps visualize the overall functionality and makes it easier to understand the system's working.

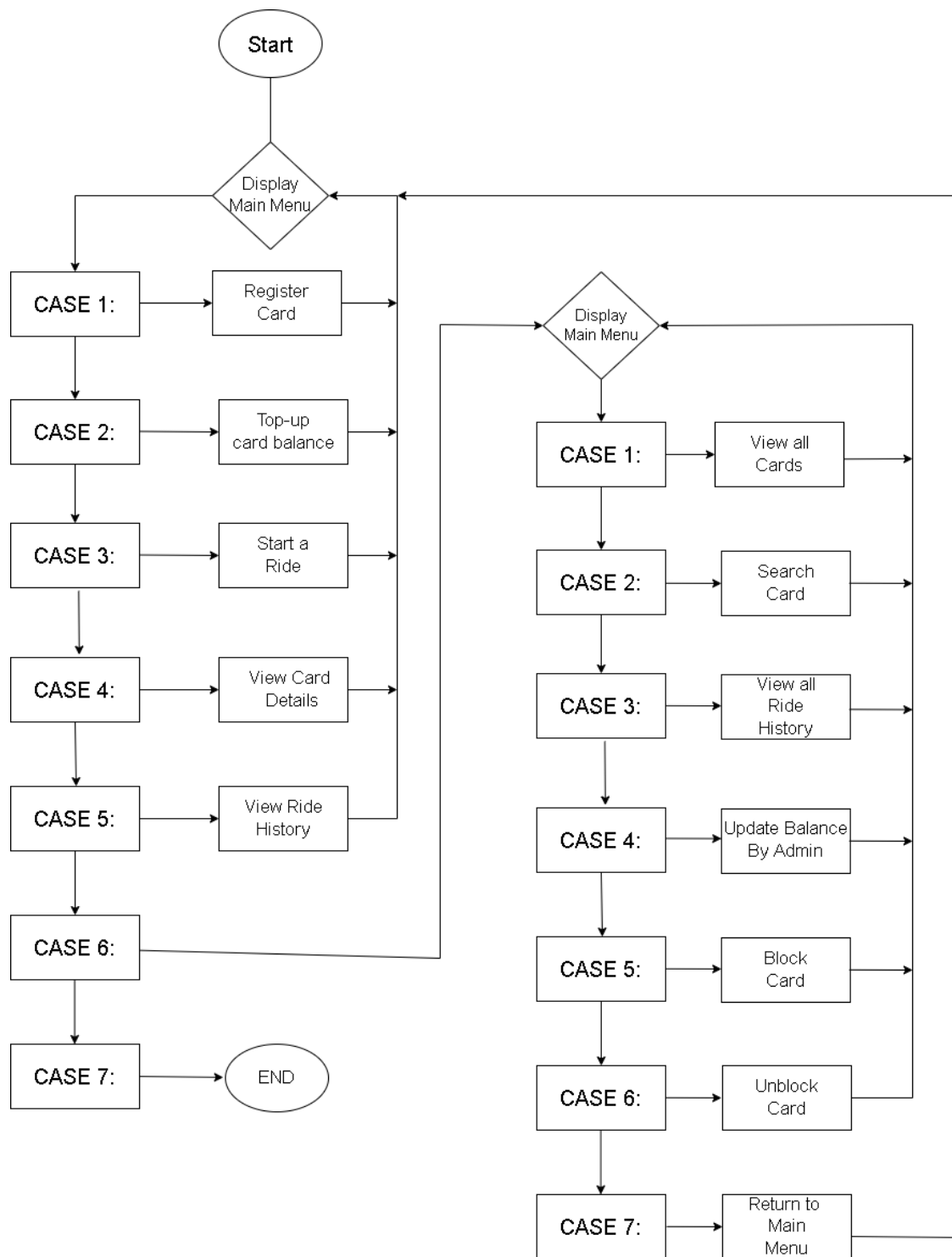


Figure 3.3 Flowchart

3.4 SYSTEM REQUIREMENT

3.4.1 FUNCTIONAL REQUIREMENT

3.4.1.1 MENU

The system provides a user-friendly, menu-driven interface that allows users to navigate between key functionalities such as Register new cards, managing rides, and top-up cards. Each menu option leads to a specific set of features designed for ease of access and interaction.

```
=====
||          MAIN MENU          ||
=====
1. Register New Card
2. Top-up Card Balance
3. Start a Ride
4. View Card Details
5. View Ride History
6. Admin Panel
7. Exit System
=====
Enter your choice (1-7): _
```

Figure 2.4.1.1 Main Menu

3.4.1.2 Top-up balance

The top-up card balance feature allows the user to enter balance and password to update their balance. It accurately shows the new balance, update in card.csv.

```
=====
Enter your choice (1-7): 2
Enter your card ID: 2082003
Card found. Details:

Card ID: 2082003
Name: pasang tamang
Type: student
Balance: Rs. 0
Expiry: 2026-12-13
Status: active
=====

Enter amount to topUP: 500
Press TAB to show or hide password
Password: *****

Enter Password: kacbit
Top-up successful! New Balance: 500
=====
```

Figure 3.4.1.3 Top-up Balance

3.4.1.3 Start rides

This system allows users to choose (tap) start stop and choose (tap) end stops so that system automatically calculates the number of stops and calculate fare accordingly.

```
Available stops:
1 kalanki
2 balkhu
3 Ekantakuna
4 Satdobato
5 Gwarko
6 Koteswor
7 tinkune
8 Sinamangal
9 Gaushala
10 Chabahil
11 Sukedhara
12 Maharajgunj
13 balaju

Enter starting stop (1-13): 2
Enter ending stop (1-13): 3
Number of stops: 1
Your fare is: Rs. 2.5

Ride started! Rs. 2.5 deducted. Remaining balance
: Rs. 497.5
2025-12-13, 17:25:09
Ride History saved!
Card updated successfully!
```

Figure 3.4.1.3 Start Ride

3.4.1.4 Admin Panel

The admin panel allows admin to control over every users, track all users in the system and also block the inactive users so that when they want to use card again they can renew it by paying some amount.

```
-----ADMIN PANEL-----
1. View all Cards
2. Search Card by Id
3. View all Ride
4. Update Card Balance
5. Remove/Block Card
6. Unblock Card
7. Exit Admin Panel
Enter choice: █
```

Figure 3.4.1.4 Admin Panel

3.4.1.5 Viewing Cards (Users)

The viewing card feature display all cards made by the users. It reads data from the storage file and presents the cards in a numbered list for easy tracking and management.

```

=====
||          CARD DETAILS          ||
=====

Card ID: 2082001
Name: Pasang
Type: student
Balance: Rs. 172.5
Expiry: 2026-10-29
Status:
=====

Card ID: 2082002
Name: Pasang Dorje Tamang
Type: student
Balance: Rs. 5117.5
Expiry: 2026-12-03
Status:
=====

```

Figure 3.4.1.5 View all Cards

3.4.1.6 Blocking and Unblocking Cards

```

-----ADMIN PANEL-----
1. View all Cards
2. Search Card by Id
3. View all Ride
4. Update Card Balance
5. Remove/Block Card
6. Unblock Card
7. Exit Admin Panel
Enter choice: 5
Enter Id to Block: 2082001
Card Blocked!
Press Enter to continue...

-----ADMIN PANEL-----
1. View all Cards
2. Search Card by Id
3. View all Ride
4. Update Card Balance
5. Remove/Block Card
6. Unblock Card
7. Exit Admin Panel
Enter choice: 6
Enter Id to Unblock: 2082001
Card UNBlocked!
Press Enter to continue...

```

Figure 3.4.1.6 Blocking and Unblocking Cards

3.4.1.7 Ride history

The ride history allows admin to view all Rides of all cards, whereas users can specifically view and track their Ride history using card ID

```
=====
||          RIDE HISTORY          ||
=====
Card ID: 2082001
Name: Pasang Tamang
Type: student
Route: kalanki -> tinkune
Fare: Rs. 15
Date: 2025-10-29
Time: 10:57:18
=====
Card ID: 2082001
Name: Pasang
Type: student
Route: kalanki -> Chabahil
Fare: Rs. 22.5
Date: 2025-10-29
Time: 11:01:28
=====
Card ID: 2082001
Name: Pasang
Type: student
Route: Satdobato -> Koteshwor
Fare: Rs. 5
Date: 2025-10-30
Time: 08:14:14
=====
```

Figure 3.4.1.7 Ride History (Admin view)

3.4.2 NON-FUNCTIONAL REQUIREMENT

The Smart Fare Simulation is designed not only to perform its intended functions but also to meet key non-functional requirements that enhance its overall quality, usability, and reliability:

1. Usability: The system uses a simple, menu-driven console interface that is easy to understand and use.
2. Portability: The application is developed in standard C++ and can run on multiple platforms with minimal changes.
3. Maintainability: Modular code structure makes the system easy to update and maintain.
4. Reliability: Input validation and proper file handling ensure stable and consistent system behaviour.
5. Performance: The system executes efficiently with low memory usage and fast response time.
6. Data Organization: Card and ride data are stored in organized CSV files for easy access and persistence.
7. Security: Basic password protection and card-blocking features help maintain data integrity.

CHAPTER 4: CONCLUSION

4.1 PROBLEM FACED

- Managing file operations (read, write, and update) in plain text files without data loss.
- Correctly calculating fare and handling discount for particular card type..
- Handling user input safely and avoiding input buffer issues with cin and fgets.
- Implementing reliable balance top-up and update without losing data.
- No built-in graphical interface, making UI limited to command-line formatting.
- Debugging logical errors and off-by-one mistakes took considerable time.
- Ensuring files exist before reading to avoid crashes.

4.2 LESSON LEARNED

- Importance of careful file handling to maintain data integrity in C++.
- How to use encryptions.
- Managing user input effectively to avoid common pitfalls with cin and fgets.
- The value of modular programming for organizing code logically.
- Necessity of validating user input and handling edge cases gracefully.
- How to debug and fix common logical errors like off-by-one mistakes.
- Limitations of command-line interfaces and the potential benefits of GUIs.
- Understanding platform-dependent functions and writing portable code.
- Patience and systematic testing are crucial for developing robust programs.

4.3 CONCLUSION

Working on this Smart fare simulation project has been a great learning experience for us. It gave us the chance to explore different concepts in C++ programming, especially related to file handling, modular functions, and time management. We really enjoyed building something that's practical and useful, even if it runs in a simple command-line interface.

With the support of our teachers and friends, we were able to complete the project successfully. It wasn't just about writing code — we also learned how to structure a complete program, handle errors, and think about how users will interact with it. These lessons will definitely help us in future academic projects and professional work.

4.4 RECOMMENDATION

- Implement a graphical user interface (GUI) for better user experience.
- Use a database (e.g., SQLite) instead of text files for more reliable data storage and querying.
- Add input validation to ensure values are entered correctly.
- Improve portability by avoiding platform-specific functions.
- Modularize code into separate files and use header files for better maintainability.
- Incorporate backup and restore features for event and to-do data.
- Add reminders or notification features for card expiry and balance finish.

4.5 FUTURE ENHANCEMENT

- Graphical User Interface (GUI): Develop a GUI version
- Navigation: Allow users to use the system with touch input.
- Balance Reminders: Add automatic reminders or notifications for Balance top-up.
- Card Categories: Organize cards by categories (e.g., student, normal, Elder). .
- Cloud Sync/Backup: Store data online or back up files to prevent data loss.
- Mobile App Version: Create an Android or iOS version for on-the-go access.
- Voice Input Integration: Allow users to add tasks or events using voice commands.

BIBLIOGRAPHY

- For vs code setup
<https://youtu.be/z2jDamkbBF0?si=Lb7JUuilViY0X0zq>
- Learning about new functions and syntax. Youtube, Ai powered tools, Text books.
- For Version control: git and github

